NANYANG TECHNOLOGICAL UNIVERSITY

Data Structures & Algorithms in Python: Queues

Dr. Owen Noel Newton Fernando

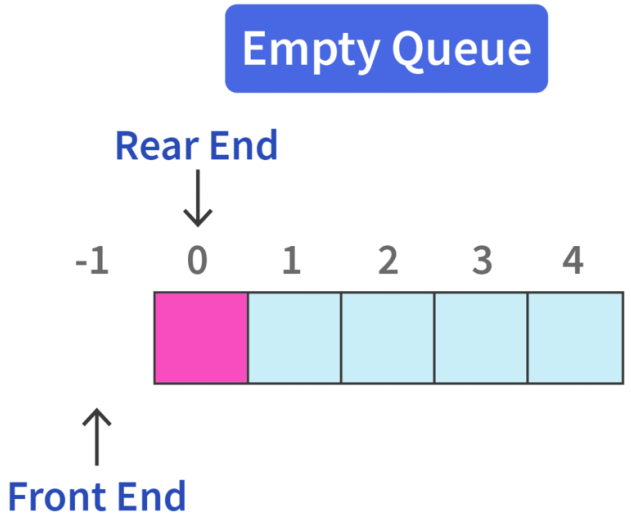
College of Computing and Data Science

Stacks, Queues, and Deques

- A stack is a last in, first out (LIFO) data structure
  - Items are removed from a stack in the reverse order from the way they were inserted
- A queue is a first in, first out (FIFO) data structure
  - Items are removed from a queue in the same order as they were inserted
- A deque is a double-ended queue--items can be inserted and removed at either end
  - Stacks and queues are special cases of deques.

Content Copyright Nanyang Technological University

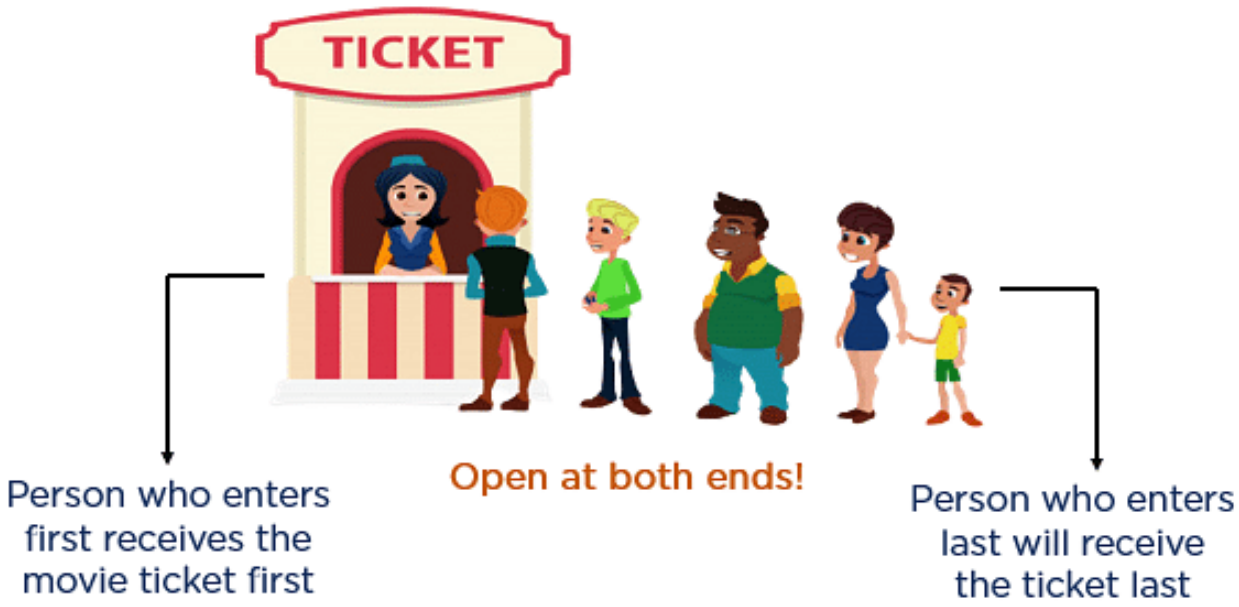
Queue – First In First Out



Content Copyright Nanyang Technological University

Examples of queues

- Physical world: movies, banks, kerosene queues.

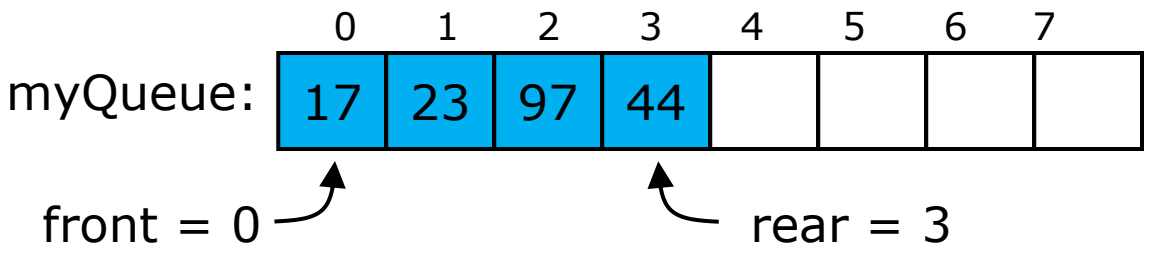


Source: simplilearn

Content Copyright Nanyang Technological University

List implementation of queues

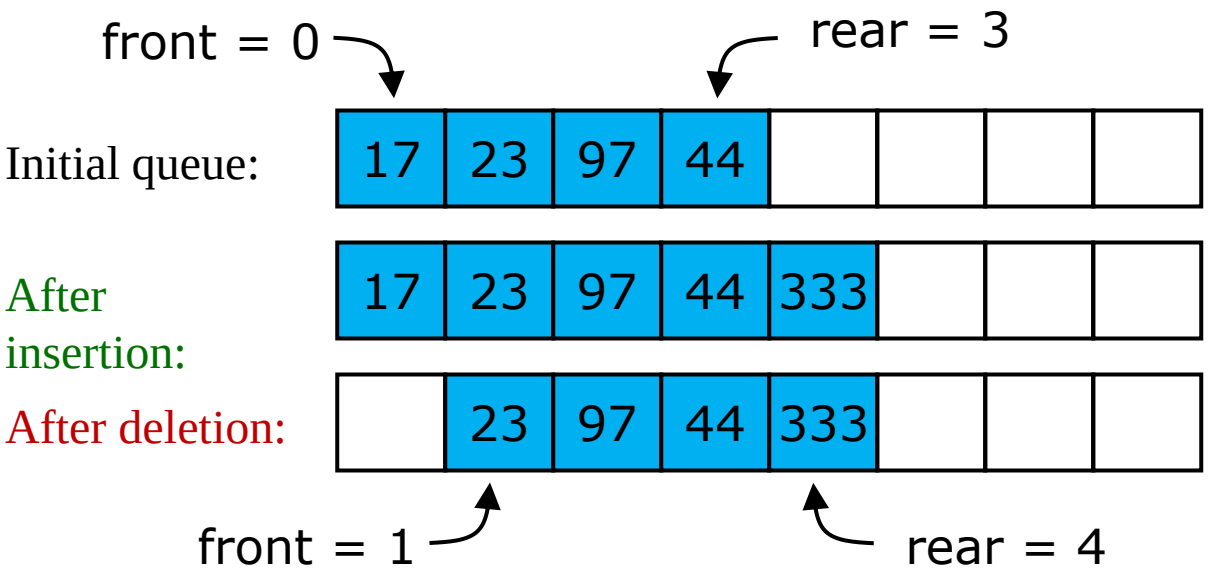
- A queue is a first in, first out (FIFO) data structure
- This is accomplished by inserting at **one end (the rear)** and deleting from the **other (the front)**



- **To insert:** put new element in location 4, and set rear to 4
- **To delete:** take element from location 0, and set front to 1

Content Copyright Nanyang Technological University

List implementation of queues

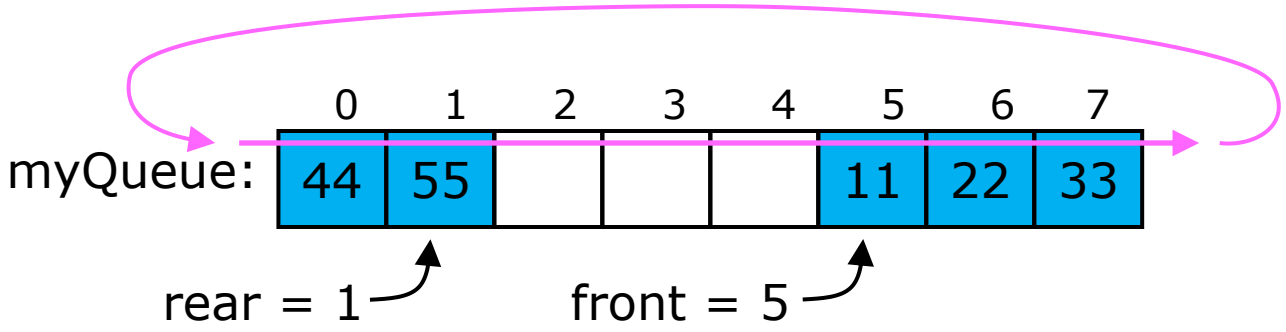


- Notice how the queue contents "crawl" to the right as elements are inserted and deleted
- What could go wrong???

Content Copyright Nanyang Technological University

Treating the internal list as circular

- We can treat the Python list holding the queue elements as circular (joined at the ends)

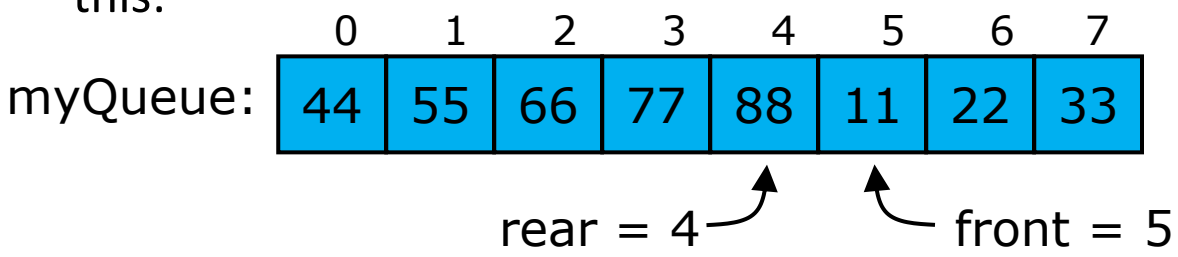


- Elements were added to this queue in the order 11, 22, 33, 44, 55, and will be removed in the same order

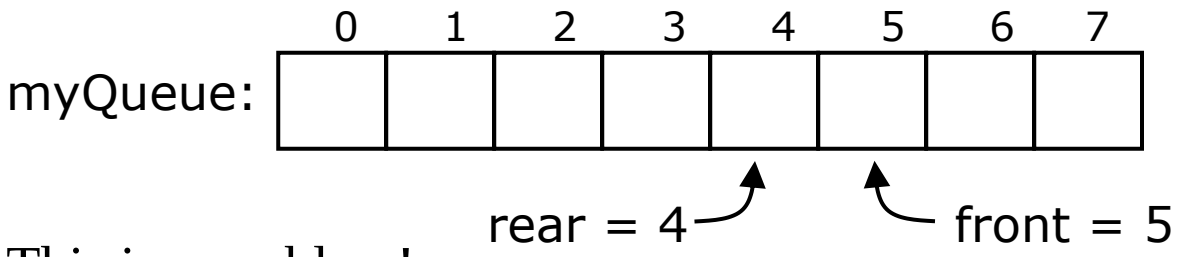
Content Copyright Nanyang Technological University

Full and empty queues

- If the queue were to become completely full, it would look like this:



- If we were then to remove all eight elements, making the queue completely empty, it could look like this:

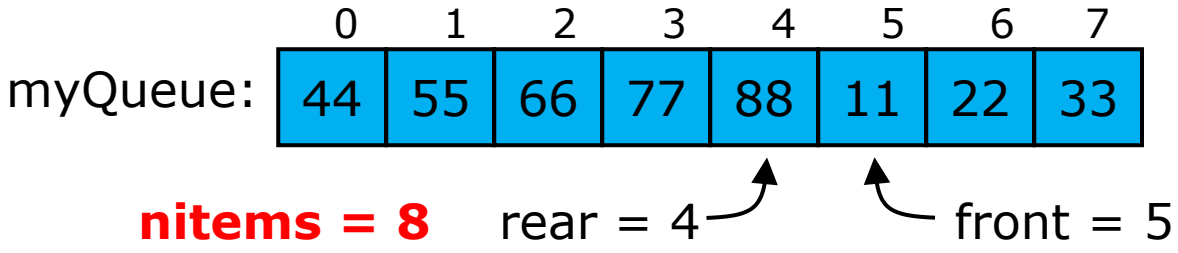


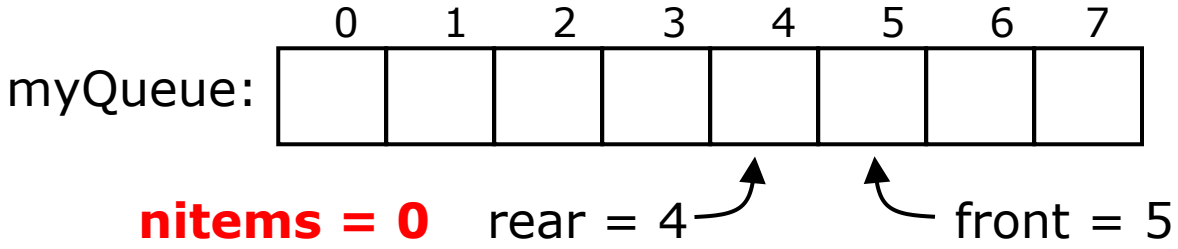
This is a problem!

Content Copyright Nanyang Technological University

Full and empty queues: solution

- Solution: Keep an additional variable (we use this approach)





Content Copyright Nanyang Technological University

Two approaches to keeping your data structure code safe

- Return a special value if overflow/underflow attempted by client code.
  - Easy to implement, but puts a burden on the caller to check the return value of such method calls.
- Throw or force an exception if overflow/underflow attempted by client code.
  - Having a method that throws an exception forces client to wrap code around every single method invocation to attempt to catch an exception.
  - Some people feel that exceptions are just glorified goto's.

Content Copyright Nanyang Technological University

A safe Queue implementation

```
class Queue(object):
   def __init__(self, size):          # Constructor
      self.__maxSize = size           # Size of [circular] array
      self.__que = [None] * size      # Queue stored as a list
      self.__front = 1                # Empty Queue has front 1
      self.__rear = 0                 # after rear and
      self.__nItems = 0               # No items in queue

   def insert(self, item):            # Insert item at rear of queue
      if self.isFull():
         raise Exception("Queue overflow")
      self.__rear += 1                # Rear moves one to the right
      if self.__rear == self.__maxSize: # Wrap around circular array
         self.__rear = 0
      self.__que[self.__rear] = item  # Store item at rear
      self.__nItems += 1
      return True

   def remove(self):  # Remove front item of queue and return it, if not empty
      if self.isEmpty():
         raise Exception("Queue underflow")
      front = self.__que[self.__front] # get the value at front
      self.__que[self.__front] = None # Remove item reference
      self.__front += 1               # front moves one to the right
      if self.__front == self.__maxSize: # Wrap around circular arr.
         self.__front = 0
      self.__nItems -= 1
      return front
```

Content Copyright Nanyang Technological University

A safe Queue implementation

```
def __init__(self, size):          # Constructor
    self.__maxSize = size          # Size of [circular] array
    self.__que = [None] * size     # Queue stored as a list
    self.__front = 1               # Empty Queue has front 1
    self.__rear = 0                # after rear and
    self.__nItems = 0              # No items in queue
```

Constructor (__init__ method)

The constructor initializes the queue with:

maxSize: The maximum number of elements the queue can hold.
que: A list to hold the queue elements.
front: The position where items are removed; starts at **index 1.**
rear: The position where items are inserted; starts at **index 0.**
nitems: A counter to keep track of the number of items in the queue.

Content Copyright Nanyang Technological University

A safe Queue implementation

```
def insert(self, item):              # Insert item at rear of queue
    if self.isFull():
        raise Exception("Queue overflow")
    self.__rear += 1                 # Rear moves one to the right
    if self.__rear == self.__maxSize: # Wrap around circular array
        self.__rear = 0
    self.__que[self.__rear] = item   # Store item at rear
    self.__nItems += 1
    return True
```

Insert Method (insert)

This method adds an item to the end (rear) of the queue:

First checks if the queue is full using isFull. If it is, an **exception** is raised.
Advances rear to the next position using **wraparound** if necessary (sets rear to 0 if it reaches the end of the queue).
Stores the new item at the updated rear position and **increments nitems by 1.**

Content Copyright Nanyang Technological University

A safe Queue implementation

```
def remove(self):  # Remove front item of queue and return it, if not empty
    if self.isEmpty():
        raise Exception("Queue underflow")
    front = self.__que[self.__front] # get the value at front
    self.__que[self.__front] = None # Remove item reference
    self.__front += 1               # front moves one to the right
    if self.__front == self.__maxSize: # Wrap around circular arr.
        self.__front = 0
    self.__nItems -= 1
    return front
```

Remove Method (remove)

This method removes and returns the item at the front of the queue:

Checks if the queue is empty using isEmpty. If it is, raises an **exception**.
Retrieves the item at the front position and then sets that position in que to None to release the reference.
Advances front to the next position, using **wraparound** if needed, and **decreases nitems by 1.**

Content Copyright Nanyang Technological University

Queue client demonstration

```
def __main():
    q = Queue(1000)

    for i in range(10, 50, 10):
        q.insert(i)

    for i in range(3):
        q.remove()

    for i in range(50, 90, 10):
        q.insert(i)

    while not q.isEmpty():
        print(str(q.remove()), end=" ")

    print()

if __name__ == '__main__':
    __main()
```

Content Copyright Nanyang Technological University

Queue implementation details

- With a list implementation:
  - you can have both overflow and underflow
  - you should set deleted elements to **None**
    - Which makes the Queue garbage collection friendly

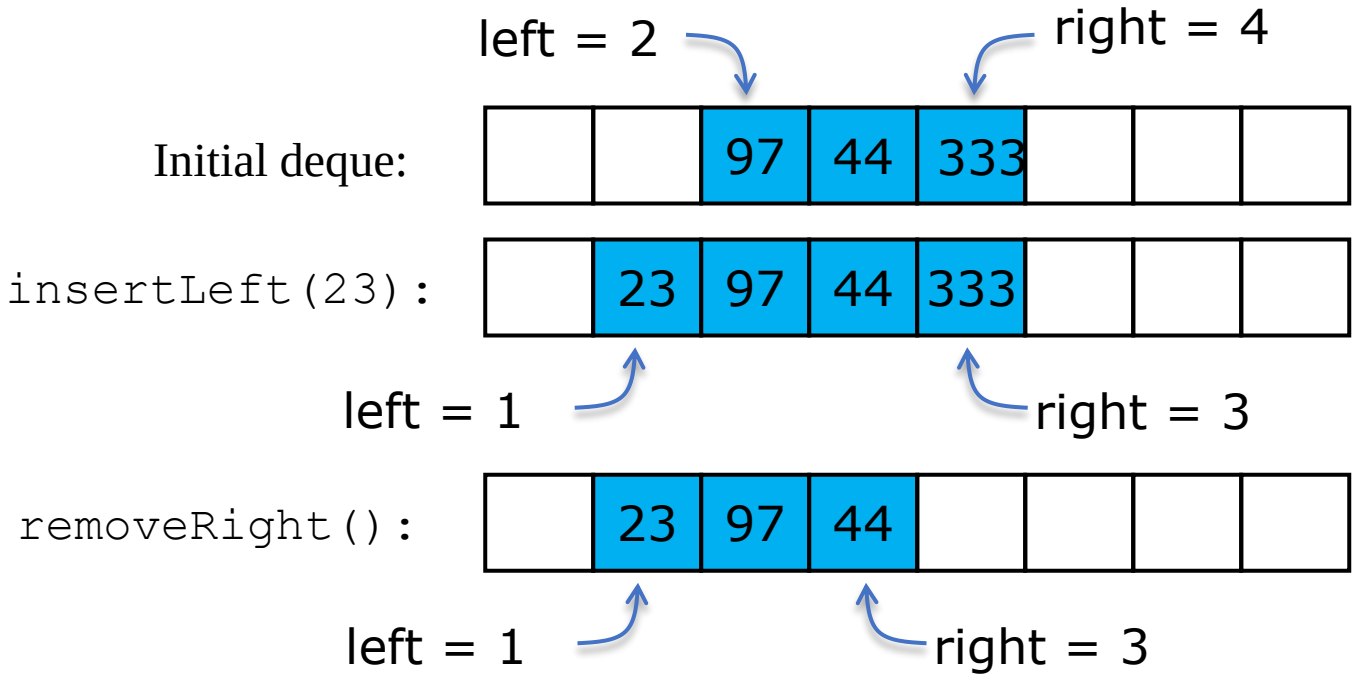
Content Copyright Nanyang Technological University

Deques

- A deque is a double-ended queue
- Insertions *and* deletions can occur at *either* end:
  - insertLeft(item)
    - Returns True or False if Deque is already Full
  - insertRight(item)
    - Returns True or False if Deque is already Full
  - removeLeft()
    - Returns either the object on the left or None if Deque is empty
  - removeRight()
    - Returns either the object on the right or None if Deque is empty
- Implementation is similar to that for queues
- Both stacks and queues can be implemented in terms of a subset of a deque's methods.

Content Copyright Nanyang Technological University

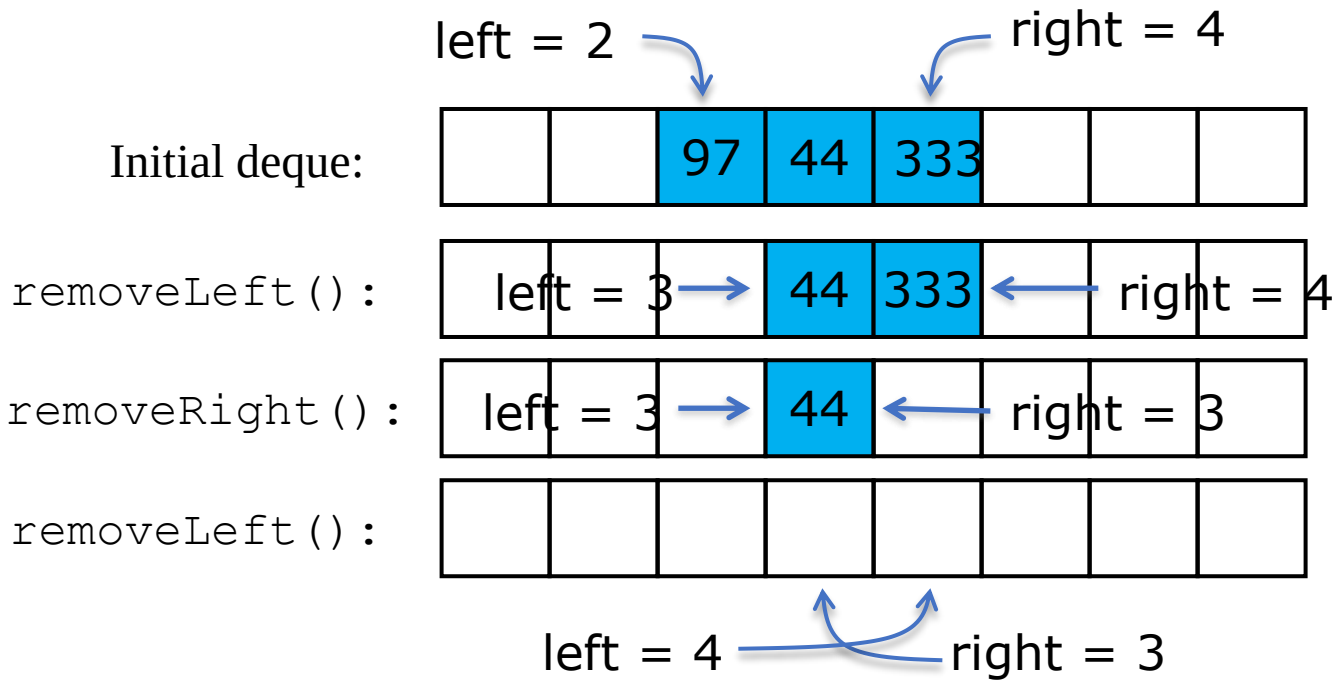
List implementation of deques



- "left" and "right" can move back and forth
- Array is treated as circular, just like with queues

Content Copyright Nanyang Technological University

List implementation of deques



- Just like queues, when deque is empty or full, “right” is exactly one to the left of “left”. Need an **`nItems`** attribute to eliminate ambiguity.
- The center is a good starting place for a deque

Content Copyright Nanyang Technological University

Priority Queues

- Stack: LIFO
- Queue: FIFO
- Priority Queue: least value first out.
  - Can implement with an array, but it can be slow.
  - There are far more efficient data structures for priority queues, but for small amounts of data a list implementation should be fine.

Content Copyright Nanyang Technological University

Array implementation performance

A priority queue could be implemented as an *unordered* **`Array`**

Adding an element would take O(1) time (why?)
Removing an element would take O(n) time (why?)
Hence, adding *and* removing an element takes O(n) time
This is a very inefficient representation

A priority queue could be implemented as effectively an *ordered* **`Array`**

Adding an element would take O(n) time (why?)
Removing an element would take O(1) time (why?)
Hence, adding *and* removing an element takes O(n) time
Again, this is very inefficient

Content Copyright Nanyang Technological University

Priority Queue implementation

```
class PriorityQueue(object):
   def __init__(self, size, pri=identity): # Constructor
      self.__maxSize = size           # Size of array
      self.__que = [None] * size      # Queue stored as a list
      self.__pri = pri                # Func. to get item priority
      self.__nItems = 0               # no items in queue

   def insert(self, item):               # Insert item at rear of
      if self.isFull():                  # priority queue if not full
         raise Exception("Queue overflow")
      j = self.__nItems - 1              # Start at front
      # Look for place by priority
      while j >= 0 and (self.__pri(item) >= self.__pri(self.__que[j])):
         self.__que[j+1] = self.__que[j] # Shift items to front
         j -= 1                          # Step towards rear
      self.__que[j+1] = item             # Insert new item at rear
      self.__nItems += 1
      return True

   def remove(self):                   # Return front item of priority
      if self.isEmpty():                 # queue, if not empty, & remove
         raise Exception("Queue underflow")
      self.__nItems -= 1                 # One fewer item in queue
      front = self.__que[self.__nItems] # Store frontmost item
      self.__que[self.__nItems] = None  # Remove item reference
      return front
```

Content Copyright Nanyang Technological University

Priority Queue implementation

```
class PriorityQueue(object):
    def __init__(self, size, pri=identity): # Constructor
        self.__maxSize = size           # Size of array
        self.__que = [None] * size      # Queue stored as a list
        self.__pri = pri                # Func. to get item priority
        self.__nItems = 0               # no items in queue
```

Constructor (__init__ method):

Initializes the priority queue with a specified maximum size (size), an optional priority function (pri), and sets up an empty queue.

Attributes:

maxSize: Stores the maximum number of items that the queue can hold.
que: A list initialized to None values to hold the queue elements.
pri: A function used to determine the priority of an item. If not provided, defaults to a simple identity function (assumes the item itself is the priority).
nitems: Tracks the current number of items in the queue, initialized to 0.

Content Copyright Nanyang Technological University

Priority Queue implementation

```
def insert(self, item):                    # Insert item at rear of
    if self.isFull():                      # priority queue if not full
        raise Exception("Queue overflow")
    j = self.__nItems - 1                  # Start at front
    # Look for place by priority
    while j >= 0 and (self.__pri(item) >= self.__pri(self.__que[j])):
        self.__que[j+1] = self.__que[j]    # Shift items to front
        j -= 1                             # Step towards rear
    self.__que[j+1] = item                 # Insert new item at rear
    self.__nItems += 1
    return True
```

insert Method:

Inserts an item into the priority queue based on its priority.

Process:

First checks if the queue is full. If it is, an exception is raised to indicate "**Queue overflow**."

The method starts from the last inserted item (**j = self.__nitems - 1**) and moves backwards to find the correct position for the new item based on its priority.
While there are still items in the queue (indicated by **j >= 0**) and the priority of the new item is greater than or equal to the priority of the item at index j, it shifts items one position to the right (to maintain order).
Once the correct position is found, the new item is inserted, and the count of items **(__nitems**) is incremented by 1.

Content Copyright Nanyang Technological University

Priority Queue implementation

```
def remove(self):                    # Return front item of priority
    if self.isEmpty():                # queue, if not empty, & remove
        raise Exception("Queue underflow")
    self.__nItems -= 1                # One fewer item in queue
    front = self.__que[self.__nItems] # Store frontmost item
    self.__que[self.__nItems] = None  # Remove item reference
    return front
```

remove Method:

Removes and returns the front item (the item with the highest priority) from the priority queue.

Process:

Checks if the queue is empty. If it is, an exception is raised to indicate "**Queue underflow**."
The front item is accessed from the list at index self.__nitems which is decremented by 1 first.
The reference to the removed item is set to None, effectively removing it from the queue.

Content Copyright Nanyang Technological University

Uses of Queues

Queues are versatile data structures with various use cases across different domains. Here are some common scenarios where queues are particularly useful:

1.Task Scheduling:

1. **Operating Systems**: Queues manage processes in CPU scheduling and task management, where processes are executed in the order they arrive (FIFO).
2. **Print Spooling**: Print jobs are managed in a queue, allowing them to be processed in the order they were submitted.

2.Breadth-First Search (BFS):

1. In graph traversal algorithms like BFS, queues are used to explore nodes level by level. Nodes are added to the queue as they are discovered and removed as they are processed.

3.Data Buffering:

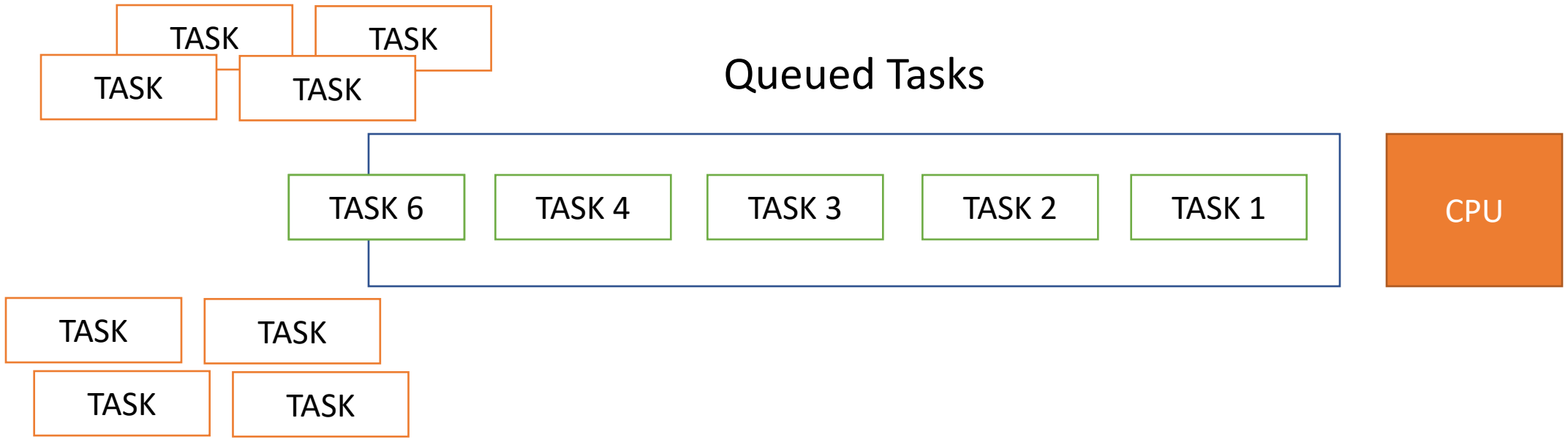
1. **Streaming Data**: Queues can buffer data streams (e.g., video/audio streaming), allowing data to be processed as it arrives.
2. **Network Data Packets**: Queues help manage incoming network packets, ensuring they are processed in the order they are received.

4.Event Management:

1. In GUI applications, events (like user interactions) can be managed using queues, allowing them to be handled in the order they occur.

Content Copyright Nanyang Technological University

Uses of Queues



Content Copyright Nanyang Technological University